Caddy Image Gallery

Operator Documentation

A Caddy v2 module for serving image and video galleries with a unified lightbox, configurable sort/pagination, and a templatable UI.

Generated June 18, 2026

<!– Comment moved here so it doesn't render as LaTeX text in the output. The TOC command below generates the actual TOC. –>

Contents

Configuration

Caddyfile directive

```
handle_path /images/* {
    root * /var/www/html/images
    image_gallery
    file_server
}
```

The `image_gallery` directive accepts one inline option:

Subdirective	Value	Default	Purpose
`template`	file name, relative to the templates dir	`gallery.tmpl`	Pick which template file to render. Path-traversal protected: no `..`, no absolute paths — the templates dir is a chroot.
`no_thumbs`	`true` / `false` (no-arg = `true`)	`false` (thumbs on)	Skip on-the-fly WebP thumbnail generation. Tile `<img src>` points to the original file instead of `~/_thumbs/<name>.webp`. Thumb requests fall through to the next handler. Useful for small galleries where you don't want a thumb cache. See `no_thumbs` walkthrough below.
`page_size`	integer >= 1	`50`	How many image entries to show per page. Must be a positive integer; `page_size 0` is rejected (use no directive, or set the explicit value you want). The pagination nav only renders when total pages > 1, so a 30-image gallery at the default 50 shows all 30 on a single page with no nav.

Example with a themed subdir:

```
handle_path /images/* {
    root * /var/www/html/images
    image_gallery {
        template themes/dark/gallery.tmpl
    }
    file_server
}
```

This loads `$GALLERY_TEMPLATES_DIR/themes/dark/gallery.tmpl` (or falls back to the bundled template if the file doesn't exist on disk). The path is validated at Provision — an invalid name (e.g. `../etc/passwd`) fails Caddy startup, not at first request.

Example: skip thumbnail generation

```
handle_path /images/* {
    root * /var/www/html/images
    image_gallery {
        no_thumbs
    }
    file_server
}
```

With `no_thumbs`: - Each tile's `<img src>` is the original image file (`./photo.jpg`), not `~/_thumbs/photo.webp` - No thumb generation, no cache, no CPU cost on first request - The browser downloads the full image per tile (bigger page payload, slower on dirs of large photos) - Requests to `~/_thumbs/<name>.webp` fall through to the next handler (file_server), which 404s

Best for: small galleries (< 100 images) where you don't want a thumb cache and the originals aren't huge. Not recommended for large galleries — the page payload goes from ~30 KB (with thumbs) to ~5 MB average (full images) for a 1,000-image dir.

Use `no_thumbs false` to turn it back on (the default is off, so the directive is opt-in). Videos are unaffected either way (they don't have thumbs — they show a play-button overlay).

Example: change the page size

```
handle_path /images/* {
    root * /var/www/html/images
    image_gallery {
        page_size 100
    }
    file_server
}
```

This shows 100 image entries per page instead of the default 50. Tradeoffs: larger pages mean fewer HTTP requests, but each request returns a bigger HTML payload (and the server uses more memory per render). The pagination nav at the bottom of the page only renders when total pages > 1, so if your gallery has 30 images and you set `page_size 100`, you get all 30 on one page with no nav. URL query override: append `?page=2` to the gallery URL to jump to a specific page. `?page_size=N` is NOT a query param — page size is set in the Caddyfile only (per-request override would let the user request arbitrarily large pages and could DOS the server).

All other configuration (the `root *` for the image directory, the `handle` / `handle_path` for the route, the auth wrapper) is via the surrounding Caddyfile block, not the `image_gallery` directive. The module reads the on-disk directory from Caddy's per-request `root` variable, or from the `Root` JSON field if set explicitly.

JSON config (advanced)

Equivalent JSON, if you're configuring Caddy via the admin API or a config file rather than a Caddyfile:

```
{
  "handler": "image_gallery",
  "root": "/var/www/html/images"
}
```

The `Root` field is optional — the module falls back to the per-request `root` set by the surrounding `handle` / `handle_path` block. Set it explicitly only if you need to override the request-time root.

Environment variables

The plugin reads one environment variable:

`GALLERY_TEMPLATES_DIR`

Where it's read in code: `render.go`, in two functions — `loadTemplate()` (at request time, to find the on-disk override) and `writeBundledTemplates()` (at Caddy startup, to write the bundled templates so operators can see them). Both do the same thing:

```
dir := os.Getenv("GALLERY_TEMPLATES_DIR")
if dir == "" {
    dir = "/etc/caddy/gallery-templates"
}
```

Default: `/etc/caddy/gallery-templates`. Created on first Caddy startup by `writeBundledTemplates()` with mode 0*755*, owned by whatever user the Caddy systemd service runs as. If the template file already exists, it's left alone (operator overrides are preserved across restarts).

The directory it points to: the absolute path you set. The plugin does not create parent directories beyond the templates dir itself — it just `MkdirAll`s the final directory. So if you point it at `/srv/gallery-templates`, that path needs to be writable by the Caddy process.

How to set it for the live Caddy (the canonical way — systemd starts the process with a clean environment, so your shell's env doesn't reach the service):

```
# One-off (persists in the manager's env, inherited by all units)
sudo systemctl set-environment GALLERY_TEMPLATES_DIR=/etc/caddy/gallery-templates
sudo systemctl restart caddy

# Or persistently in the caddy service unit
sudo systemctl edit caddy
# Add:
#   [Service]
#   Environment="GALLERY_TEMPLATES_DIR=/etc/caddy/gallery-templates"
# Or use EnvironmentFile= pointing at a file with the line
```

```
sudo systemctl daemon-reload
sudo systemctl restart caddy
```

How to set it for dev (`go test`, `xcaddy build`, or running a custom Caddy from your shell):

```
export GALLERY_TEMPLATES_DIR=/path
# then run your commands
```

Note: the test suite sets `GALLERY_TEMPLATES_DIR` to a temp dir via `TestMain`, so `go test` is isolated from your shell's value. The module always reads the env at request time, so changes to the env var take effect on the next Caddy restart — no rebuild needed.

What happens if the dir is unwritable or doesn't exist: `writeBundledTemplates()` logs a warning to stderr and continues. The bundled templates still serve fine (the on-disk file is a convenience, not a requirement). The next request falls back to the bundled constant.

There are no other env vars. Sort order, pagination, page size, and the thumb cache directory are all configurable in code (constants in the source) rather than via env vars.

In-code constants (not configurable)

These are baked into the source. They can be changed by editing the code, rebuilding, and restarting Caddy — but they're not exposed as Caddyfile directives or env vars. Listed here so you know what the defaults are and where to find them.

Constant	Value	Where
Thumbnail size	320px	`thumbnails.go`
Thumb `Cache-Control` max-age	86400 (24h)	`thumbnails.go`
Thumb format	WebP (lossless, via `nativewebp`)	`thumbnails.go`
Scan cache TTL	1 minute	`gallery.go` (`NewScanCache(time.Minute)`)
Thumb width (px)	320	`thumbnails.go` (the resize)

Note: "Thumbnail size" and "Thumb width (px)" both reference the same constant (320) — the maximum dimension of the generated thumbnail. If you wanted them to be different values (say, max-dim 320 and width 280 for a non-square crop), let me know and I'll split them into two separate constants.

Why these aren't configurable yet: most operators never need to change them. The thumb size and Cache-Control max-age are sensible defaults that work for the vast majority of galleries. The scan cache TTL is short enough (1 minute) that new files appear quickly without manual cache busting, and long enough that hot directories don't re-scan on every request. If you find yourself wanting to change one of these, that's a signal that the constant should probably be promoted to a config option — file an issue and we can discuss.

What `image_gallery` does NOT do

- It does not generate thumbnails at build time — thumbnails are generated on-the-fly on first request and cached in `/var/cache/caddy-gallery/`. See the wiki page for cache details.
- It does not handle uploads or modifications. It's read-only.

- It does not support nested directory pagination — when you click into a subdirectory, that subdirectory has its own gallery with its own pagination, but the parent dir's image list isn't merged in.
- It does not redirect from `/foo` to `/foo/`. If you request `/images/generated` (no trailing slash) and it's a directory, the module returns a 301 with a relative `Location: generated/` header (so the browser resolves it relative to the current URL). This is a workaround for Caddy's `handle_path` rewriting both `r.URL.Path` and `r.RequestURI`, which makes absolute reconstruction impossible inside the handler.

caddy_image_gallery — Complete configuration reference

A single-page reference for every configuration knob the `image_gallery` Caddy handler exposes. Use this as the "what can I configure" reference; for how to use individual features, see the per-topic docs linked below.

For per-topic deep dives: - configuration.md — Caddyfile directive details + env vars - templates.md — the template file + variables - sort-and-pagination.md — the URL query API

---

1. Caddyfile subdirectives (the primary way)

Inside an `image_gallery { ... }` block:

Subdirective	Value	Default	Purpose
`sort`	`mtime` / `name` (also accepts `date` as an alias for `mtime` — see Sort & Pagination to Aliases)	`mtime` (newest first)	Default sort field. Overridable per-request via `?sort=`.
`template`	file name, relative to the templates dir	`gallery.tmpl`	Which template file to render. Path-traversal protected.
`no_thumbs`	no-arg = `true` / explicit `false` = `false`	`false` (thumbs on)	Skip thumbnail generation. Tile `<img>` points to the original file.
`page_size`	integer >= 1	`50`	Image entries per page. Nav only renders when `total pages > 1`.
`thumb_width`	integer >= 1	`320`	Max width in pixels. Source is fit-within-bounds.
`thumb_height`	integer >= 1	`320`	Max height in pixels. Source is fit-within-bounds.
`thumb_format`	`jpeg` / `jpg` / `png` / `webp`	`webp` (lossless)	Output format. jpeg quality 75, png lossless, webp lossless.

Subdirective	Value	Default	Purpose
`cache_scan`	integer >= 1	`1`	Scan cache TTL in minutes.
`thumb_ttl`	integer >= 1	`1440`	HTTP `Cache-Control: max-age` for thumbs, in minutes (= 24h default).

Full Caddyfile example (every directive set):

```
handle_path /images/* {
    root * /var/www/html/images
    image_gallery {
        sort name
        template themes/dark/gallery.tmpl
        no_thumbs false
        page_size 100
        thumb_width 480
        thumb_height 320
        thumb_format jpeg
        cache_scan 5
        thumb_ttl 60
    }
    file_server
}
```

---

2. JSON config fields (programmatic)

Same fields, json tags. Use `caddy adapt` or any JSON config producer to set them:

```
{
  "root": "/var/www/html/images",
  "sort": "name",
  "template": "themes/dark/gallery.tmpl",
  "no_thumbs": false,
  "page_size": 100,
  "thumb_width": 480,
  "thumb_height": 320,
  "thumb_format": "jpeg",
  "cache_scan": 5,
  "thumb_ttl": 60
}
```

`cache` is runtime-only (excluded from JSON via `json:"-"`).

---

3. Per-request URL query parameters (no config needed)

Param	Values	Default	Effect
`sort`	`mtime` / `name` / `type` / `size` (also accepts `date` as an alias for `mtime` — see Sort & Pagination to Aliases)	inherits from Caddyfile	Sort field
`order`	`asc` / `desc`	depends on `sort`	Sort direction
`page`	integer >= 1	`1`	Which page (only meaningful when `page_size` causes pagination)

Deliberately NOT query-overridable: - `?page_size=N` — would let users request arbitrarily large pages and could DOS the server - `?thumb_format=N` — would let users force the server to recompute thumbs in any format - `?thumb_width=N` — same

The page size, format, and thumb dimensions are set in the Caddyfile only.

---

4. Environment variables

Var	Default	Purpose
`GALLERY_TEMPLATES_DIR`	`/etc/caddy/gallery-templates`	Where the module looks for (and writes) the on-disk template override. The only user-facing env var.

A second env var exists in the code for testing only: `GALLERY_THUMB_CACHE_DIR` (default `/var/cache/caddy-gallery`) — not documented as a user-facing knob.

See configuration.md to Environment variables for the full setup story (systemd unit, dev workflow, failure mode).

---

5. Caddyfile handler-level wiring (external, not `image_gallery`-specific)

These live in the surrounding Caddyfile, not inside the `image_gallery { ... }` block:

Caddyfile construct	Purpose	Example
`handle_path` (or `route`)	Strips URL prefix and routes to `image_gallery`	`handle_path /images/* { ... }`
`root *`	Per-request "root" var, read by `image_gallery` to find the directory to scan	`root * /var/www/html/images`
`file_server`	Fallthrough for direct file requests and 404s (must come AFTER `image_gallery`)	`file_server`
Auth (Authelia, basic_auth, etc.)	Wraps the route in auth	`(auth)` snippet (or `basicauth { ... }` block)

6. External: the Caddyfile as a whole

The live site typically looks like:

```
hermes.synapticloop.com {
    import auth
    route /images/* {
        root * /var/www/html/images
        image_gallery
        file_server
    }
}
```

The `(auth)` snippet is defined elsewhere in the Caddyfile and wraps every route in Authelia `forward_auth` at `127.0.0.1:9091`.

---

7. Behavior modifiers (not configurable, hardcoded)

These are baked into the source. They could be promoted to config options if a use case comes up, but currently aren't:

Constant	Value	Where
Thumb JPEG quality (when `thumb_format jpeg`)	75	`thumbnails.go`
Video thumbs	not generated (videos show play-button overlay in the template)	`thumbnails.go` (source extensions are image-only)
Lightbox	page-scoped (50 images on the current page, not all 1,197)	`render.go` (lightbox JS in the template)
Other-files strip on a subdir page	"Other files" rendered as horizontal chips above the image grid	`render.go` (splitFiles)
Directories strip on every page	Always rendered, alphabetical, ignores sort	`render.go` (splitFiles + dirs sort)

---

8. Quick reference — what's where

If you're wondering "where is this knob?":

Want to configure…	Use
Sort field (default + per-request)	`sort` directive + `?sort=` query
Pagination	`page_size` directive + `?page=` query
Thumbnail on/off	`no_thumbs` directive
Thumbnail size	`thumb_width` / `thumb_height` directives
Thumbnail format (jpeg/png/webp)	`thumb_format` directive

Want to configure...	Use
Thumbnail browser cache TTL	`thumb_ttl` directive
Directory scan cache TTL	`cache_scan` directive
Template file (theme)	`template` directive (e.g. `themes/dark/gallery.tmpl`)
Templates directory (where to find templates)	`GALLERY_TEMPLATES_DIR` env var
Thumbnails cache directory	`GALLERY_THUMB_CACHE_DIR` env var (testing only)
Auth (who can view)	Caddyfile `basicauth` or `(auth)` snippet (external)
Route (URL prefix)	Caddyfile `handle_path` or `route` (external)
Image directory	Caddyfile `root *` (external)

Templates

The gallery is rendered by an `html/template` (Go's standard template engine). The template is bundled in the binary as a Go string constant (`galleryTemplate` in `render.go`), and on each Caddy startup the module also writes it to disk at `/etc/caddy/gallery-templates/gallery.tmpl` (or `$GALLERY_TEMPLATES_DIR` if set). The on-disk file exists so operators can read the template without opening the binary, and so they can override individual templates by editing the file in place.

How the template loading works

```
loadTemplate(name string) reads $GALLERY_TEMPLATES_DIR
    │
    ├── name = gallery.Template (from Caddyfile `template` directive)
    │   default "gallery.tmpl" if the directive is absent
    │
    ├── sanitizeTemplateName(name):
    │   reject absolute paths, reject ".." (any traversal)
    │
    ├── /etc/caddy/gallery-templates/<clean name> exists?
    │     ├── YES to parse that file directly (CSS+JS are inside it)
    │     │         (the on-disk file is the active template)
    │     └── NO  to use the bundled galleryTemplate constant
    │
    └── return *template.Template ready to RenderPage
```

The `template` Caddyfile directive is the operator-facing knob that picks the template file. See `docs/configuration.md` for the directive syntax and the path-traversal protection details.

Note on `no_thumbs`: the template is unaffected by the `no_thumbs` Caddyfile directive. The template always uses `{{.ThumbURL}}` for the tile `<img src>`; the `no_thumbs` flag changes the *value* of that field (to the original file URL when true, to the thumb URL when false), not the field name or its usage. So a template that works with thumbs also works with no_thumbs (and vice versa), with no template changes needed.

The template is a single self-contained file. The HTML, CSS (inside `<style>`), and JS (inside `<script>`) all live in one Go string constant (`galleryTemplate` in `render.go`) and one on-disk file (`/etc/caddy/gallery-templates/gallery.tmpl`). There is no sub-template loading — the previous

Phase 16 design that split them into 3 files (`gallery.tmpl` + `style.css` + `lightbox.js`) was collapsed in Phase 17 for easier editing.

The bundled constant is the source of truth at build time. The on-disk file is an *override* layer; if you delete it, the module falls back to the bundled version automatically on the next request.

What's in the single template file

One file, `gallery.tmpl`, ~16.7 KB / 574 lines, containing:

Section	What's in it	Approx lines
`<!DOCTYPE>` … `<style>`	Document head, including the full CSS inlined as a `<style>` block	~10 + 365 (CSS)
`<body>`, `<main>`, `<header>`	Title, counts, sort indicator, sort bar	~50
Directories section	`{{if .Directories}}` … `{{end}}` — the dirs chip strip	~7
Other files section	`{{if .OtherFiles}}` … `{{end}}` — the others chip strip	~7
Images section	Sort info, paginated grid, per-tile HTML	~80
Pagination	`{{if gt .TotalPages 1}}` … `{{end}}` — prev/next links	~15
`<script>` … `</script>`	The full JS inlined (lightbox, open-in-new-tab, sort indicator)	~100 (JS)

The CSS rules and the HTML they apply to are interleaved top-to-bottom in the file, so when you scroll through the template you see the structure (HTML), the styling (CSS), and the behavior (JS) in document order. Easier to hand-edit than three separate files.

Template variables

The template receives a `PageData` struct. All fields you can reference from `{{...}}` are listed below.

Top-level fields (on `PageData`)

Field	Type	What it's for
`{{.Title}}`	`string`	The page title (rendered in `<title>` and the `<h1>`). Currently the directory name.
`{{.PathPrefix}}`	`string`	Prefix for relative links to files in the same directory. Usually `"./"`.
`{{.ThumbPrefix}}`	`string`	Prefix for thumbnail URLs. Usually `"./_thumbs/"`.

Field	Type	What it's for
`{{.Directories}}`	`[]FileView`	Subdirectory entries. Always rendered in full (not paginated), case-insensitive alphabetical.
`{{.OtherFiles}}`	`[]FileView`	Non-media files (HTML, txt, etc.). Always rendered in full (not paginated).
`{{.Images}}`	`[]FileView`	The image + video tiles for the current page only. Paginated, sorted per the user's `?sort=&order=` choice.
`{{.Page}}`	`int`	Current page number (1-based).
`{{.PageSize}}`	`int`	Images per page (constant; default 50).
`{{.TotalImages}}`	`int`	Total images in the directory (across all pages).
`{{.TotalPages}}`	`int`	Total page count.
`{{.HasPrev}}`	`bool`	True if `{{.Page}} > 1`.
`{{.HasNext}}`	`bool`	True if `{{.Page}} < {{.TotalPages}}`.
`{{.Sort}}`	`SortSpec`	The current sort. See below.

`{{.Sort}}` (a `SortSpec` struct)

Field	Type	What it's for
`{{.Sort.Field}}`	`string`	One of `"mtime"`, `"name"`, `"type"`, `"size"`. (The `?sort=` URL param. `"mtime"` is the default.)
`{{.Sort.Order}}`	`string`	`"asc"` or `"desc"`. (The `?order=` URL param.)

Per-entry fields (on `FileView`)

When you `{{range .Images}}` (or `.Directories` / `.OtherFiles`), each iteration gives you a `FileView` with these fields:

Field	Type	What it's for
`{{.Name}}`	`string`	The basename (`"photo.jpg"`, `"subdir"`, etc.). Truncated with ellipsis in the live template if too long for the tile.
`{{.Href}}`	`string`	Relative link to the file. Use as `<a href="{{.Href}}">`.

Field	Type	What it's for
`{{.ThumbURL}}`	`string`	For images and videos, the relative thumbnail URL (e.g. `./_thumbs/photo.webp`). Empty string for non-media files — check `{{if .ThumbURL}}` before using.
`{{.IsDir}}`	`bool`	True for directories.
`{{.IsImage}}`	`bool`	True for image files.
`{{.IsVideo}}`	`bool`	True for video files. Videos go in the image grid with a play-button overlay (no `<img>` child); `{{.ThumbURL}}` is set but the live template doesn't render an `<img>` for them.
`{{.IsOther}}`	`bool`	True for non-media files (HTML, txt, etc.).
`{{.Type}}`	`string`	Uppercase extension without the dot: `"JPG"`, `"DIR"`, `"MP4"`, `"HTML"`, etc.
`{{.Size}}`	`string`	Human-readable file size: `"234 KB"`, `"1.2 MB"`, etc. Empty string for directories.
`{{.Date}}`	`string`	ISO date `"2006-01-02"` (UTC-normalised). Empty string for directories.

Template functions (the funcmap)

The template engine has a few helper functions registered:

Func	Signature	What it does
`minus1`	`minus1 n int to int`	Returns `n - 1`. Used for prev-page link targets.
`plus1`	`plus1 n int to int`	Returns `n + 1`. Used for next-page link targets.
`sortLabel`	`sortLabel field string to string`	Maps a sort field code to its display label: `"name"to"Name"`, `"type"to"Type"`, `"mtime"to"Modified"`, `"size"to"Size"`, `"date"to"Date"`. Unknown fields fall back to the raw field name capitalised. Empty string to `"Modified"` (the default).

Editing the templates — the basics

1. Find the templates dir. On this host it's `/etc/caddy/gallery-templates/`. The file is `gallery.tmpl`.

2. Edit the file in place. The module re-reads it on every request, so changes take effect immediately — no Caddy restart needed.
3. Test in a browser. The next request will use the new template. If the template has a parse error, the module will return 500 with the parser error in the response body. Fix the syntax and try again.
4. Revert to the bundled version. If you want to go back to what the binary ships with, delete the on-disk file: `sudo rm /etc/caddy/gallery-templates/gallery.tmpl`. The next request falls back to the bundled constant.

Walkthrough: change the theme to dark mode

Since the CSS is now inlined in the same file as the HTML, you edit the `<style>` block directly in `gallery.tmpl` — no second file to coordinate, no sub-template indirection.

1. Open `/etc/caddy/gallery-templates/gallery.tmpl` in your editor.
2. Find the `<style>` block. It's the big CSS block near the top of the file, just after `<title>...</title>`. Search for `html, body { background: #f3f6f7;` to find the body color.
3. Change the colors inline. The CSS is plain CSS — no preprocessing. Most of the theme colors are concentrated in the first 50-100 lines.
4. Save the file. The change takes effect on the next request (no Caddy restart needed — the module re-reads the on-disk template on every `loadTemplate` call).
5. Hard-reload in the browser to bypass the HTML cache.

A common set of swaps for dark mode (find/replace these in the `<style>` block):

Light (default)	Dark variant
`html, body { background: #f3f6f7; }`	`html, body { background: #1a1a1a; color: #ddd; }`
`main { background: white; }`	`main { background: #222; }`
`header { border-bottom: 1px solid #e5e9ea; }`	`header { border-bottom: 1px solid #333; }`
`.chip { background: #f3f6f7; border: 1px solid #e5e9ea; }`	`.chip { background: #2a2a2a; border: 1px solid #3a3a3a; }`
`a { color: #006ed3; }`	`a { color: #4ea3ff; }`

Walkthrough: add a "Created" column to the image tiles

If you want to show the file's created time alongside the modified date (the template currently shows `Date` which is the modified date):

1. Note: `FileView.Date` is the only date field exposed. The `FileInfo` struct in the scanner *does* have `ModTime` (in nanoseconds) but no `CreatedTime`. Adding a "Created" column would require a code change — extend `FileView` with a `Created string` field, populate it in `RenderPage` from `info.ModTime` (or from `os.Stat` birthtime via `syscall.Statx` on Linux), then reference it in the template as `{{.Created}}`.

   In other words: this is a code change, not a template-only change. The template is fully driven by the Go struct fields — anything you want to display has to be in the struct.

Where the bundled templates are defined in source

If you want to read the source (or fork and customise):

File	Constant	Lines (approx)
render.go	galleryTemplate	line 392, ~574 lines (HTML + inlined CSS + inlined JS)

A single constant. CSS and JS are inlined inside <style> and <script> blocks respectively. To customise: edit the constant, rebuild the module (./build.sh), restart Caddy (sudo systemctl restart caddy). The on-disk template is written from the new constant on the next startup.

Upgrading from a pre-inlining install (Phase 16 to Phase 17)

If your site was running the old 3-file template split (gallery.tmpl + style.css + lightbox.js) and you upgraded to the new inlining build, the on-disk files are in an inconsistent state:

- style.css and lightbox.js from the old build are now dead weight. writeBundledTemplates removes them automatically on the next Provision after upgrade. Safe to ignore.
- The on-disk gallery.tmpl from the old build still has the old {{template "lightbox.js" .}} references, which no longer work. The new loadTemplate will fail to parse this file and the gallery will 500.

One-time fix on upgrade:

```
sudo rm /etc/caddy/gallery-templates/gallery.tmpl
sudo systemctl restart caddy
```

The next Provision writes the new inlined template. After that, the on-disk file is the canonical inlined version, and any operator edits to it become live overrides.

Upgrading the bundled template content

writeBundledTemplates() deliberately does NOT overwrite an existing gallery.tmpl on disk — that's the operator-override contract. When the bundled template's contents change (e.g. new CSS rule, new template branch, fixed layout bug), the on-disk file is NOT updated automatically.

To pick up the new bundled content:

```
# 1. Save any local customisations (if you have any)
diff /etc/caddy/gallery-templates/gallery.tmpl      /home/osmanj/projects/
    caddy_image_gallery/render.go
# 2. Delete the on-disk file
sudo rm /etc/caddy/gallery-templates/gallery.tmpl
# 3. Restart Caddy so the next Provision runs writeBundledTemplates
sudo systemctl restart caddy
# 4. (Optional) Re-apply your local customisations
```

This workflow is intentional: operators who have customised the template (e.g. themed dark mode) keep their changes across upgrades. Operators who haven't customised just get the "fresh bundled version" after the rm + restart.

Future enhancement (not v1): writeBundledTemplates could write a content hash into a sidecar file, and on Provision, if the sidecar hash doesn't match the bundled hash, overwrite the on-disk file. This would auto-update without breaking the operator-override contract (the operator could still delete the on-disk file to fall back to the bundled version, OR write a sidecar with a custom hash to pin to their version).

Troubleshooting

Edit took effect but the page looks the same. Hard-reload (Cmd-Shift-R / Ctrl-F5). The browser may have cached the previous HTML.

Edit took effect but the page is a 500. Your template has a parse error. The response body will contain the Go `html/template` parser error. Common causes: - Unclosed `{{if}}` / `{{range}}` / `{{with}}` blocks - `{{end}}` mismatch (most common — Go templates require `{{end}}` to close every block) - Calling a method that doesn't exist on the data type (e.g. `{{.Foo.Bar}}` when `Foo` is a string) - An unescaped backtick inside a Go template comparison (backticks terminate the Go raw string the constant is in)

Edit took effect but layout is broken. You removed or re-ordered a structural element. The CSS selectors and the JS `querySelector` calls expect a specific DOM shape — search for the class name in the template to see what's expected.

Want to test a new template before deploying it. Stage the edit in a new file at, say, `/etc/caddy/gallery-templates/gallery.tmpl.staging`, then copy it over the live file once you're happy. Or `curl` the gallery to see the live HTML and check it with a browser inspector before saving.

Got a 500 immediately after upgrading. You have the pre-inlining on-disk `gallery.tmpl` from a previous build. See the "Upgrading from a pre-inlining install" section above.

Sort & Pagination

The gallery's image grid respects three URL query parameters. All three are optional. They're a stable URL API — bookmarking a `?sort=size&order=desc&page=3` link works and gives the same view on every visit.

Query parameters

Param	Values	Default	What it does
`?sort=`	`name` / `type` / `size` / `mtime`	`mtime`	What field to sort images by. The URL also accepts `?sort=date` (treated as `?sort=mtime`) for back-compat. See "Aliases" below.
`?order=`	`asc` / `desc`	`desc`	Sort direction. `desc` for `mtime` is newest-first; `desc` for `name`/`type`/`size` is Z-A / largest-first.
`?page=`	integer >= 1	`1`	Which page of results to show. 1-based. Out-of-range or non-numeric values fall back to 1.

The URL parameter is preserved across sort changes: clicking "Name" on `?sort=mtime&order=desc&page=2` becomes `?sort=name&order=asc&page=2` (the toggle flips the order for the new field;

the page stays the same).

The directory strip and other-files strip are NOT affected by the sort. They always render in: - Directories: case-insensitive alphabetical (`splitFiles` re-sorts them on every render) - Other files: scanner order (same default as the image list — newest-first by mtime)

This is intentional: the dirs strip is a navigation aid and should be stable, not reshuffle when the user changes the image sort. See Phase 15 in the plan for the reasoning.

Sort indicator in the header

The header shows the current sort + a reset link:

- On the default sort (`mtime desc`), the indicator is a plain `<span>` — clicking it would just reload the same page, so it's not a link. It reads `Sort: Modified $\downarrow$`.
- On any other sort, the indicator is a link to `?` (no query params, which is the default). It reads e.g. `Sort: Size $\uparrow$`.

Sort buttons

The sort bar at the top of the header has four buttons: Name, Type, Modified, Size. Each shows a `$\uparrow$` or `$\downarrow$` arrow if it's the currently active sort. Clicking the active button toggles the order; clicking an inactive button switches to that field at the direction that's the "natural" opposite of the current order (so you don't get the same direction twice in a row).

The button labels are produced by the `sortLabel` template function (see the Templates doc for the full map). To add a new sort field:

1. Add a `case` to `sortLabel` in `render.go`
2. Add a `case` to the `sortFiles` function (the sort implementation)
3. Add a button to the `gallery.tmpl` template

It's a code change in three places; the template is just the visible button.

Pagination

50 images per page (constant `PageSize = 50` in render.go, not configurable via env var or Caddyfile — it's a code constant).

Pagination links in the live template: - The "First" and "Prev" pagination buttons (rendered by the template with double- and single-left-arrow icons, when `{{.HasPrev}}` is true) - The "Next" and "Last" pagination buttons (single- and double-right-arrow icons, rendered when `{{.HasNext}}` is true) - Current page indicator in the middle

The page numbers in the URL are 1-based, not 0-based. `?page=0` falls back to `?page=1`.

Examples

```
/images/                          # default: mtime desc, page 1
/images/?sort=name&order=asc      # alphabetical
/images/?sort=size&order=desc     # largest first
/images/?page=3                   # third page of default sort
/images/?sort=type&order=asc&page=2
```

Aliases

`?sort=date` is an alias for `?sort=mtime` — both sort by the file's modification time. The two values produce identical results:

```
/images/?sort=date&order=desc    # == /images/?sort=mtime&order=desc
/images/?sort=date&order=asc     # == /images/?sort=mtime&order=asc
```

Why have the alias? The two names describe the same thing from different angles — "mtime" is the technical field name (modification time, from the filesystem), "date" is the user-friendly name (the date the file was last changed). The "Modified" sort button in the UI emits `?sort=mtime` (the technical name), but the URL API accepts both for back-compat with bookmarks and external links that might use either name.

Implementation: `parseSort` in `render.go` accepts `"date"` in its switch statement and treats it identically to `"mtime"`:

```
switch field {
case "name", "type", "date", "mtime", "size":
    // all valid; "date" is an alias for "mtime"
default:
    field = "mtime" // unknown fields fall back to the default
}
```

In `sortFiles`, both `"mtime"`, `"date"`, and `""` (the empty default) share the same sort branch — sort by `ModTime` per the `order` param. So functionally there's no distinction between them.

When to use which: if you're writing a new integration or bookmark, prefer `?sort=mtime` (it's the canonical name and what the UI button emits). `?sort=date` is fine too — just pick one and stick with it for consistency.

Stability guarantees

- The URL parameter API is stable. Bookmarking works. The four sort fields and two orders are part of the public contract; renaming or removing one is a breaking change.
- Adding a new sort field is non-breaking (just adds a new option).
- The default sort may change in a future major version, but the `?sort=mtime` URL will always be honored.
- Page size is a code constant and may change in future versions. URLs with `?page=N` are stable relative to the *current* page size — if the page size changes from 50 to 100, page 3 of the old URL might now be page 2 of the new layout, but no image is lost or duplicated (it's the same image set, just packed differently).